RAPPORT DE DEPLOIEMENT

Application Evenix — Frontend + Backend

BTS SIO SLAM — 2025/2026

RESUME DU PROJET

Frontend React/Vite → déployé sur Vercel
Backend Spring Boot (Java) → déployé sur Render
Base de données MySQL → déployé sur TiDB Cloud

LIENS EN LIGNE

Service	Lien
Frontend (site web)	https://mon-projet-backend-frontend-spyr.vercel.app
Backend (API)	https://evenix-backend.onrender.com
API Evénements	https://evenix-backend.onrender.com/api/evenements/all

ARCHITECTURE DU PROJET

Le projet est organisé en monorepo avec deux sous-dossiers dans un seul dépôt GitHub :

Dossier	Description
Backend/	Application Spring Boot (Java 21) — API REST
Frontend/	Application React + TypeScript + Vite
Backend/Dockerfile	Configuration pour le déploiement Docker sur Render
Frontend/public/vercel.json	Configuration du routage pour Vercel

ETAPE 1 — BASE DE DONNEES (TiDB Cloud)

La base de données MySQL a été déployé sur TiDB Cloud, un service gratuit et toujours accessible depuis des serveurs externes comme Render.

1.1 Création du compte TiDB

1. Aller sur tidbcloud.com
2. Cliquer sur Sign up avec GitHub
3. Creer un nouveau cluster en choisissant Serverless (gratuit)
4. Choisir la région Europe
5. Cliquer sur Create

1.2 Récupération des informations de connexion

Une fois le cluster crée, cliquer sur Connect pour récupérer les informations :

Parametre	Valeur
HOST	gateway01.eu-central-1.prod.aws.tidbcloud.com
PORT	4000
USERNAME	3mR9zUw1KNB8X4q.root
DATABASE	evenix

1.3 Création de la base de données

Dans le SQL Editor de TiDB Cloud, exécuter les commandes suivantes :

```
CREATE DATABASE evenix;
USE evenix;
```

1.4 Création des tables

Exécuter le script SQL pour créer toutes les tables nécessaires : adresse, rôle, user, tarif, type-event, événement, réservation, sponsoriser, user-rôle.

1.5 Insertion des données

Insérer les données initiales : rôles (ADMIN, USER), utilisateurs de test, tarifs, types d'événements, adresses et événements.

Base de données déployée avec succès sur TiDB Cloud — toujours accessible 24h/24.

ETAPE 2 — BACKEND (Render)

Le backend Spring Boot a été déployé sur Render en utilisant Docker. Render est une plateforme gratuite qui supporte les applications Java.

2.1 Préparation du code

Avant le déploiement, plusieurs fichiers ont été modifiés ou crées :

Modification de application.properties

Le fichier de configuration a été modifié pour lire les paramètres de connexion depuis les variables d'environnement :

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
server.port=${PORT:8081}
```

Modification du CORS (SecurityConfig.java)

Le fichier SecurityConfig.java a été modifié pour autoriser les requetés provenant du frontend deploye sur Vercel :

```
config.setAllowedOrigins(Arrays.asList(
    "http://localhost:5173",
    "https://mon-projet-backend-frontend-spyr.vercel.app"
));
```

Creation du Dockerfile

Un fichier Dockerfile a été crée dans le dossier backend/ pour permettre le déploiement sur Render :

```
FROM maven:3.9-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY backend/ .
RUN mvn clean package -DskipTests
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
```

```
CMD ["java", "-jar", "app.jar"]
```

2.2 Déploiement sur Render

6. Aller sur render.com et se connecter avec GitHub
7. Cliquer sur New → Web Service
8. Sélectionner le dépôt GitHub -MonProjet_Backend_Frontend
9. Configurer : Language = Docker, Dockerfile Path = backend/Dockerfile
10. Choisir le plan gratuit
11. Ajouter les variables d'environnement
12. Cliquer sur Deploy

2.3 Variables d'environnement sur Render

Variable	Valeur
DB_URL	jdbc:mysql://gateway01.eu-central-1.prod.aws.tidbcloud.com:4000/evenix?useSSL=true&serverTimezone=UTC
DB_USERNAME	3mR9zUw1KNB8X4q.root
DB_PASSWORD	***** (mot de passe TiDB)

Backend deploye avec succes. Accessible sur : https://evenix-backend.onrender.com

ETAPE 3 — FRONTEND (Vercel)

Le frontend React/Vite a été déployé sur Vercel, une plateforme gratuite et optimisée pour les applications React.

3.1 Préparation du code

Création du fichier vercel.json

Un fichier vercel.json a été créé dans frontend/public/ pour gérer le routage React Router (eviter les erreurs 404 sur les routes) :

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Configuration de la variable d'environnement

Le fichier constants.ts utilise une variable d'environnement pour pointer vers le backend en ligne :

```
export const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:8081';
```

3.2 Déploiement sur Vercel

13. Aller sur vercel.com et se connecter avec GitHub
14. Cliquer sur New Project
15. Selectionner le depot -MonProjet_Backend_Frontend
16. Configurer le Root Directory = frontend (dossier du frontend)
17. Framework Preset = Vite (detecte automatiquement)
18. Ajouter la variable d'environnement VITE_API_URL
19. Cliquer sur Deploy

3.3 Variable d'environnement sur Vercel

Variable	Valeur
VITE_API_URL	https://evenix-backend.onrender.com

> Frontend déployé avec succès. Accessible sur : https://mon-projet-backend-frontend-spyr.vercel.app

ETAPE 4 — GESTION GIT

Toutes les modifications ont été poussées sur GitHub pour déclencher les déploiements automatiques.

Commandes utilisées

```
git add .
git commit -m "Description de la modification"
git push origin main
```

Chaque push sur la branche main déclenche automatiquement un nouveau déploiement sur Vercel. Pour Render, un déploiement manuel est parfois nécessaire.

COMPTES UTILISATEURS DE TEST

Email	Mot de passe	Role

admin@example.com	Admin123!	**ADMIN**
user@example.com	User123!	**USER**

PROBLEMES RENCONTRES ET SOLUTIONS

Problème 1 — Import de la base de données

Problème : La base de données Aiven s'éteignait automatiquement avec le plan gratuit. FreeSQLdatabase ne permettait pas les connexions externes depuis Render.

Solution : Migration vers TiDB Cloud qui offre un plan gratuit toujours actif et accepte les connexions depuis des serveurs externes.

Problème 2 — Erreur SSL avec TiDB

Probleme : La connexion JDBC échouait avec l'erreur SSL.

Solution : Ajout du paramètre useSSL=true dans l'URL de connexion et configuration du certificat CA.

Problème 3 — Erreur 404 sur les routes React

Problème : Les routes React comme /login donnaient une erreur 404 sur Vercel car Vercel ne connaissait pas le routage React Router.

Solution : Création d'un fichier vercel.json avec une règle de redirigèrent qui renvoie toutes les requêtes vers index.html.

Problème 4 — Erreur CORS

Problème : Le backend refusait les requêtes venant du frontend Vercel avec l'erreur "origin not allowed".

Solution : Ajout de l'URL Vercel dans la liste des origines autorisées dans SecurityConfig.java, puis redéploiement sur Render.

Problème 5 — Fichier Dockerfile

Problème : Le Dockerfile ne trouvait pas le fichier pom.xml car le chemin était incorrect.

Solution : Modification du Dockerfile pour copier le contenu du dossier backend/ et utiliser Maven directement (mvn au lieu de ./mvnw).

CONCLUSION

Le déploiement du projet Evenix a été réalisé avec succès en utilisant trois services cloud gratuits :

- TiDB Cloud pour la base de données MySQL — toujours disponible, compatible avec Spring Boot
- Render pour le backend Spring Boot — déploiement via Docker, plan gratuit
- Vercel pour le frontend React/Vite — déploiement automatique depuis GitHub

L'application est entièrement accessible en ligne et peut être présentée au jury sans nécessite d'une installation locale.